МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

федеральное государственное автономное образовательное учреждение высшего образования «Самарский национальный исследовательский университет имени академика С.П. Королева»

(Самарский университет)

Институт информатики и кибернетики

Кафедра информационных систем и технологий

Отчет по практической работе №1
по курсу
«Нейронные сети и машинное обучение»

Выполнила:

студент группы 6231-090401D

Ризванова Э. Т.

Проверила:

к.т.н., доцент Солдатова О.П.

Самара 2025

СОДЕРЖАНИЕ

Введение 3

1 Порядок выполнения работы 4

2 Выполнение задания 5

3 Метод k-ближайших соседей 8

4 Наивный байесовский классификатор 11

Заключенние 14

Приложение А. Листинг программы 15

ВВЕДЕНИЕ

Целью практической работы «Исследование эффективности моделей машинного обучения для решения задачи классификации на примере решения задачи классификации трёх различных сортов итальянских вин» является демонстрация способности моделей машинного обучения решать задачи классификации.

Необходимо реализовать модель на языке Python в среде Jupyter Notebook с использованием библиотек NumPy, pandas, Matplotlib, scikit-learn.

Используемые модели машинного обучения приведены в таблице 1.

Таблица 1 – Вариант задания для выполнения практической работы 1

№	Класс 1	Класс 2	Классификатор 1	Классификатор 2
10	0	1	К-ближайших соседей (К=5)	Наивный байесовский классификатор

1 Порядок выполнения работы

1) считать файл с данными для классификации итальянских вин, встроенный в библиотеку scikit-learn;

2) построить диаграмму рассеивания для заданных классов вин;

3) построить классификаторы для заданной пары классов итальянских вин в соответствии с вариантом задания. Обучить и протестировать классификаторы;

4) рассчитать метрики accuracy, precision, recall, TPR, FPR, F1;

5) построить ROC кривые и посчитать площадь AUC под кривой;

6) рассчитать и построить графическое изображение для Confusion matrics;

7) сравнить метрики и ROC-кривые. Сделать вывод о выборе лучшего классификатора по критерию F-меры и площади AUC. В качестве ответа в текстовое поле написать наилучший классификатор по каждому из критериев;

8) продемонстрировать работу моделей машинного обучения преподавателю;

9) составить отчёт, который должен содержать постановку задачи, текст программы, результаты работы программы по пунктам 2-8 и сдать его преподавателю.

2 Выполнение задания

Загрузка данных из датасета с помощью функции load_wine из библиотеки scikit-learn показана на рисунке 1.

```
data = load_wine()
X = data.data
Y = data.target
feature_names = data.feature_names

print('Количество образцов:', X.shape[0])
print('Количество признаков:', X.shape[1])
print('Названия признаков:', feature_names)
print('Матрица признаков:')
print(X)
print('Метки:')
print(Y)
```

✓ 0.0s

```
Количество образцов: 178
Количество признаков: 13
Названия признаков: ['alcohol', 'malic_acid', 'ash', 'alcalinity_of_ash', 'magnesium', 'total_phenols', 'flavanoids',
Матрица признаков:
[[1.423e+01 1.710e+00 2.430e+00 ... 1.040e+00 3.920e+00 1.065e+03]
 [1.320e+01 1.780e+00 2.140e+00 ... 1.050e+00 3.400e+00 1.050e+03]
 [1.316e+01 2.360e+00 2.670e+00 ... 1.030e+00 3.170e+00 1.185e+03]
 ...
 [1.327e+01 4.280e+00 2.260e+00 ... 5.900e-01 1.560e+00 8.350e+02]
 [1.317e+01 2.590e+00 2.370e+00 ... 6.000e-01 1.620e+00 8.400e+02]
 [1.413e+01 4.100e+00 2.740e+00 ... 6.100e-01 1.600e+00 5.600e+02]]
Метки:
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2]
```

Рисунок 1 – Загрузка набора данных

Подготовим данные для построения диаграммы рассеивания. Создание DataFrame, а также разделение данных на признаки и метки класса представлено на рисунке 2.

```
df = pd.DataFrame(X, columns=feature_names)
df['target'] = Y

df_filtered = df[df['target'].isin([0, 1])]
X_filtered = df_filtered.drop('target', axis=1)
Y_filtered = df_filtered['target']

print('Количество образцов после фильтрации:', df_filtered.shape[0])
print('Количество образцов класса 0:', sum(Y_filtered == 0))
print('Количество образцов класса 1:', sum(Y_filtered == 1))
```

✓ 0.0s

```
Количество образцов после фильтрации: 130
Количество образцов класса 0: 59
Количество образцов класса 1: 71
```

Рисунок 2 – Подготовка данных

Диаграмма рассеивания для всех пар признаков с помощью функции pairplot из пакета seaborn показана на рисунке 3.

```
sns.pairplot(df_filtered, hue='target', palette={0:'red', 1:'blue'}, vars
plt.show()
```

✓ 47.7s Py

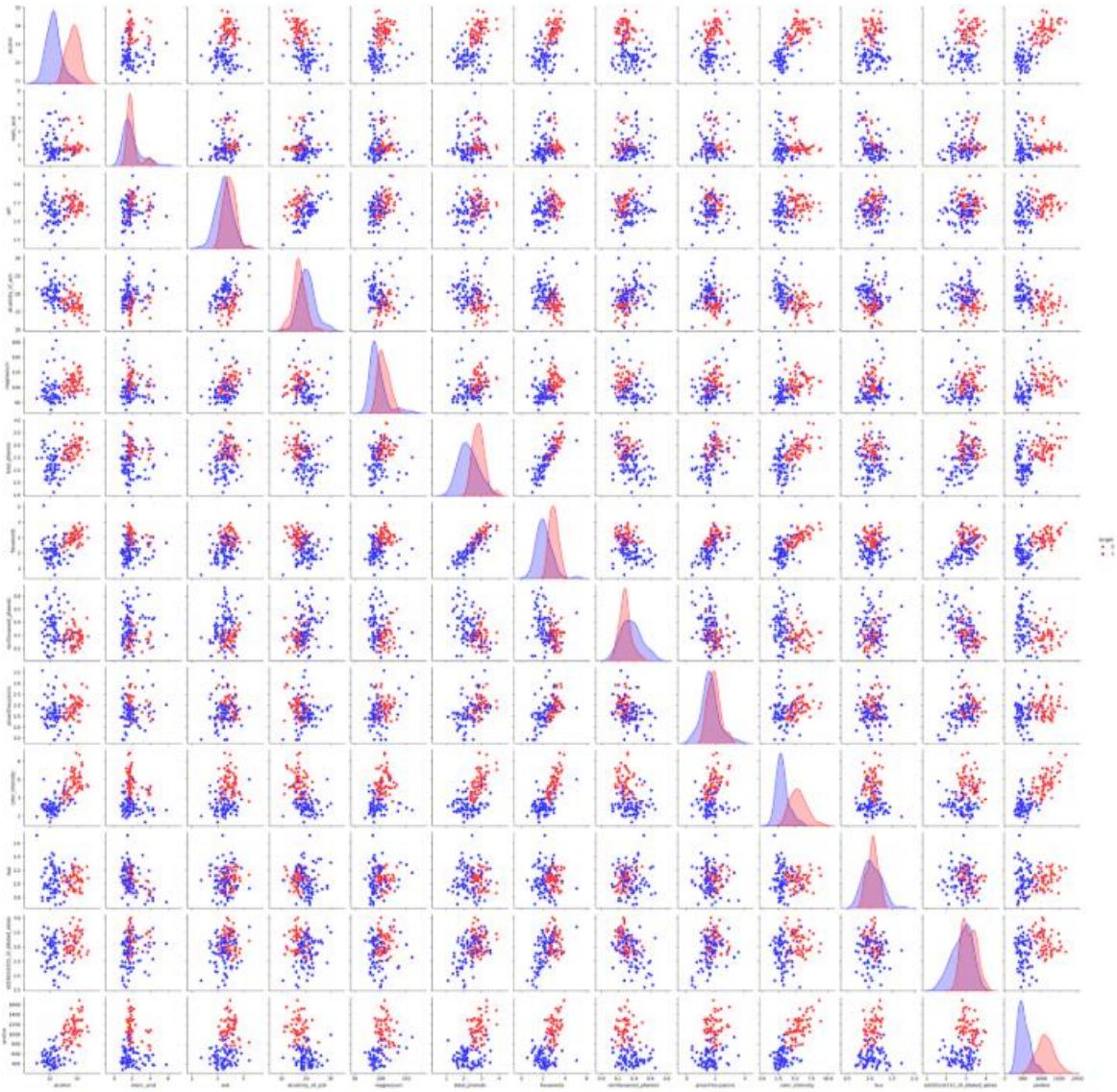

Рисунок 3 – Диаграмма рассеивания для всех пар признаков

На представленных диаграммах рассеивания можно выделить несколько ключевых аспектов анализа. Например, в распределении «proline» видно, что красные точки имеют более высокие значения, чем синие. Это указывает на то, что между двумя группами существуют значительные различия в содержании «proline». Некоторые точки могут находиться далеко от основной массы данных, что указывает на возможные выбросы. В некоторых

диаграммах видно, что между переменными существует положительная или отрицательная корреляция. Например, если точки образуют восходящую линию, это указывает на положительную корреляцию, как в случае с «total_phenols» и «flavanoids».

Приступим к написанию классификаторов. Разделение выборки на обучающую и тестовую с помощью функции train_test_split из пакета sklearn.model_selection представлено на рисунке 4.

```
X_train, X_test, Y_train, Y_test = train_test_split(X_filtered, Y_filtered, test_size=0.4, random_state=42)

print('Количество образцов в обучающей выборке:', X_train.shape[0])
print('Количество образцов в тестовой выборке:', X_test.shape[0])
✓ 0.0s
```

```
Количество образцов в обучающей выборке: 78
Количество образцов в тестовой выборке: 52
```

Рисунок 4 – Разделение выборки

Произведем стандартизацию данных для улучшения точности классификации. Стандартизация – это метод, который приводит данные к нулевому среднему значению и единичному стандартному отклонению. Стандартизация данных показана на рисунке 5.

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print('Среднее по каждому признаку:', X_train.mean(axis=0))
print('Отклонение по каждому признаку:', X_train.std(axis=0))
✓ 0.0s
```

```
Среднее по каждому признаку: [-4.76826555e-16 -5.33761070e-17  2.42683366e-15  1.38635542e-15
  3.18833279e-16  2.74353190e-16 -6.73962310e-16 -4.02989607e-17
  3.82884607e-16  6.47630098e-16  4.61169564e-16  2.72858659e-15
  1.72226905e-16]
Отклонение по каждому признаку: [1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1.]
```

Рисунок 5 – Стандартизация данных

3 Метод k-ближайших соседей

Классификатором 1 является метод k-ближайших соседей. Метод k-ближайших соседей (K-Nearest Neighbors, KNN) – это метод решения задачи классификации, который определяет принадлежность объекта к тому классу, которому принадлежит большинство из k ближайших к нему объектов в пространстве признаков. Число k определяет количество ближайших объектов, с которыми сравнивается классифицируемый элемент. Как правило, значение k выбирают нечётным, чтобы избежать неоднозначности при голосовании.

Для реализации KNN будем использовать библиотеку scikit-learn. Ниже представлен код для решения задачи классификации вин, аналогичной той, что выполняли ранее.

На рисунке 6 представлено, как выглядит обучение k-ближайших соседей (k = 5) в коде.

```
knn_clf = KNeighborsClassifier(n_neighbors=5)
knn_clf.fit(X_train, Y_train)

Y_predict_knn = knn_clf.predict(X_test)
print(classification_report(Y_test, Y_predict_knn, digits=3))
```

✓ 0.0s

```
              precision    recall  f1-score   support

           0      0.900     1.000     0.947        27
           1      1.000     0.880     0.936        25

    accuracy                          0.942        52
   macro avg      0.950     0.940     0.942        52
weighted avg      0.948     0.942     0.942        52
```

Рисунок 6 – Метод k-ближайших соседей (k = 5)

Прогнозы классификатора 1 на тестовой выборке показывают результаты, приведённые в таблице 2.

Таблица 2 – Результаты прогноза классификатора 1 на тестовой выборке

Accuracy	Precision:	Recall	F1
0,942	0,950	0,940	0,942

Построим ROC кривую и посчитаем площадь AUC под ней для оценки качества бинарного классификатора. График ROC кривой для классификатора 1 представлен на рисунке 7.

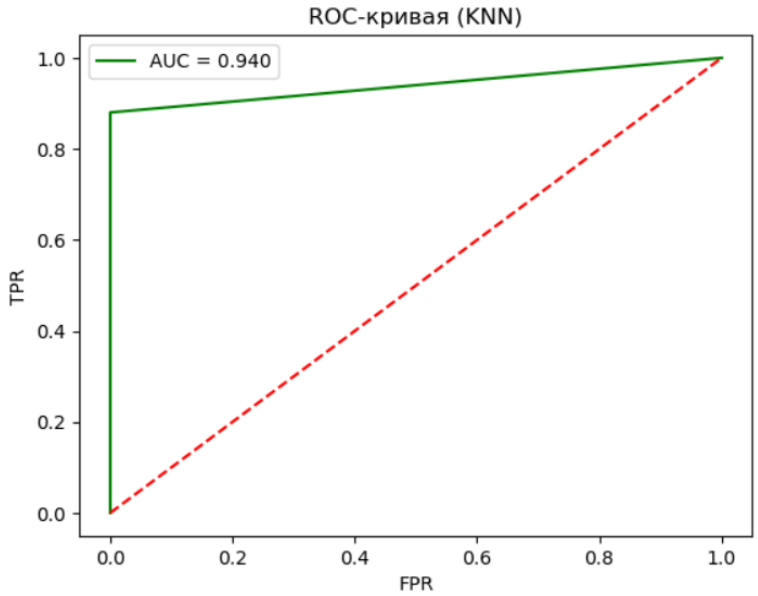

Рисунок 7 – График ROC кривой для классификатора 1

Построим матрицу ошибок для оценки качества работы классификатора. Она представляет собой таблицу, которая позволяет визуализировать производительность модели путем сравнения предсказанных классов с истинными классами. Матрица ошибок для классификатора 1 представлена на рисунке 8.

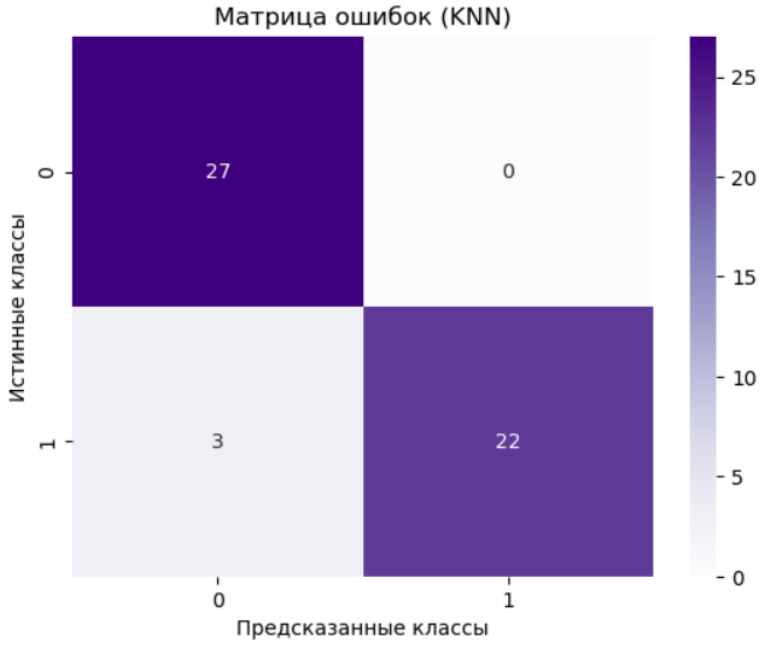

Рисунок 8 – Матрица ошибок для классификатора 1

По данной матрице можно сказать, что классификатор 1 верно классифицировал 27 положительных и 22 отрицательных случаев. Неправильно предсказаны 3 положительных и 0 отрицательных случаев.

4 Наивный байесовский классификатор

Классификатором 2 является наивный байесовский классификатор. Наивный байесовский классификатор – простой вероятностный классификатор, основанный на применении теоремы Байеса со строгими (наивными) предположениями о независимости.

Предположения:

- известна функция правдоподобия $P(x \mid y)$;

- известны априорные вероятности $P(y), P(x)$.

Используем теорему Байеса. Пусть события $H_1, H_2, \dots, H_n$ образуют полную группу. Пусть событие А наступило ($P(A) > 0$). Тогда вероятность того, что при этом была реализована гипотеза H_k вычисляется по формуле:

$$P\left(\frac{H_k}{A}\right) = \frac{P(H_k)\mathrm{P}\left(\frac{\mathrm{A}}{H_k}\right)}{P(A)} = \frac{P(H_k)P\left(\frac{A}{H_k}\right)}{\sum_{i=1}^{n} P(H_i)P\left(\frac{A}{H_i}\right)}.$$

Формула Байеса позволяет переоценить вероятности гипотез после того, как проведено испытание, в результате которого произошло событие А.

Для реализации наивного байесовского классификатора используем библиотеку scikit-learn. Код для решения задачи классификации вин представлен на рисунке 9.

```
nb_clf = GaussianNB()
nb_clf.fit(X_train, Y_train)

Y_predict_nb = nb_clf.predict(X_test)

print(classification_report(Y_test, Y_predict_nb, digits=3))
✓ 0.0s
```

```
              precision    recall  f1-score   support

           0      0.963     0.963     0.963        27
           1      0.960     0.960     0.960        25

    accuracy                          0.962        52
   macro avg      0.961     0.961     0.961        52
weighted avg      0.962     0.962     0.962        52
```

Рисунок 9 – Наивный байесовский классификатор

Прогнозы классификатора 2 на тестовой выборке показывают результаты, приведённые в таблице 3.

Таблица 3 – Результаты прогноза классификатора 2 на тестовой выборке

Accuracy	Precision:	Recall	F1
0,962	0,961	0,961	0,961

При равных условиях значения метрик классификатора 2 больше, чем у первого.

По графику ROC кривой, построенном для классификатора 2, видно, что этот классификатор имеет значение AUC, более близкое к 1 (рисунок 10).

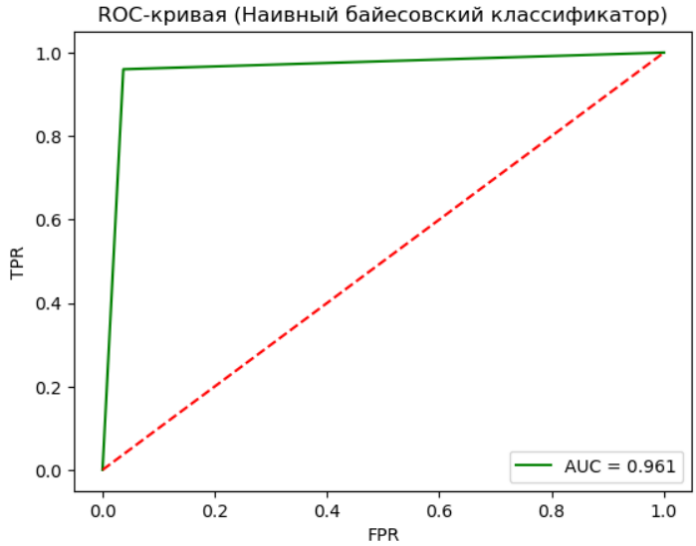

Рисунок 10 – График ROC кривой для классификатора 2

Матрица ошибок для классификатора 2 представлена на рисунке 11.

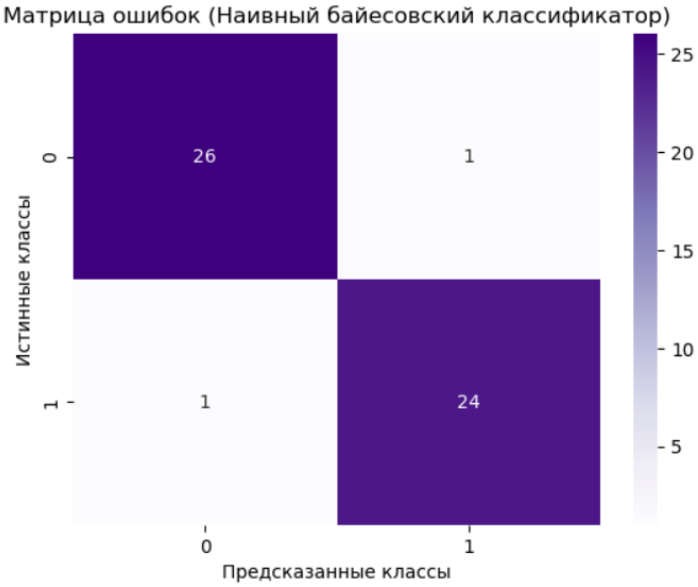

Рисунок 11 – Матрица ошибок для классификатора 2

По матрице ошибок видно, что второй классификатор ошибался в предсказании меньше, чем первый. Классификатор 2 верно классифицировал 26 положительных и 24 отрицательных случаев. Неправильно предсказаны 1 положительных и 1 отрицательных случаев.

Сравнение классификаторов представлено на рисунке 12.

```
f1_knn = f1_score(Y_test, Y_predict_knn)
f1_nb = f1_score(Y_test, Y_predict_nb)

if f1_knn > f1_nb:
    print("По критерию F1 лучше KNN.")
else:
    print("По критерию F1 лучше Наивный байесовский классификатор.")

if auc_knn > auc_nb:
    print("По критерию AUC лучше KNN.")
else:
    print("По критерию AUC лучше Наивный байесовский классификатор.")
```

✓ 0.0s

```
По критерию F1 лучше Наивный байесовский классификатор.
По критерию AUC лучше Наивный байесовский классификатор.
```

Рисунок 12 – Сравнение классификаторов

По итогам сравнения наивный байесовский классификатор стал лучшим по критериям F1 и AUC.

ЗАКЛЮЧЕННИЕ

В рамках работы были реализованы два классификатора: k-ближайших соседей (k = 5) и наивный байесовский классификатор. Обучение и тестирование проводилось на данных библиотеки scikit-learn.

На первом этапе был загружен набор данных о сортах итальянских вин. Данные включают 13 признаков для каждого образца вина. Для задачи классификации были выделены два класса. Кроме того, была построена диаграмма рассеивания по нескольким признакам для визуального анализа.

Для реализации моделей использовались классификаторы из библиотеки scikit-learn. Были рассчитаны метрики: accuracy, precision, recall, F1, TPN, FPN, а также построены ROC-кривые и вычислены значения AUC.

Модели были протестированы на тестовом наборе данных. По результатам экспериментов, наивный байесовский классификатор показал более высокие результаты и оказался предпочтительнее для задачи классификации сортов вин.

ПРИЛОЖЕНИЕ А

Листинг программы

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split

from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB

from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, roc_curve, auc, confusion_matrix, f1_score

data = load_wine()
X = data.data
Y = data.target
feature_names = data.feature_names

print('Количество образцов:', X.shape[0])
print('Количество признаков:', X.shape[1])
print('Названия признаков:', feature_names)
print('Матрица признаков:')
print(X)
print('Метки:')
print(Y)

df = pd.DataFrame(X, columns=feature_names)
df['target'] = Y

df_filtered = df[df['target'].isin([0, 1])]
X_filtered = df_filtered.drop('target', axis=1)
Y_filtered = df_filtered['target']

print('Количество образцов после фильтрации:', df_filtered.shape[0])
print('Количество образцов класса 0:', sum(Y_filtered == 0))
print('Количество образцов класса 1:', sum(Y_filtered == 1))

sns.pairplot(df_filtered, hue='target', palette={0:'red', 1:'blue'}, vars=feature_names)
plt.show()

X_train, X_test, Y_train, Y_test = train_test_split(X_filtered, Y_filtered, test_size=0.4,
random_state=42)

print('Количество образцов в обучающей выборке:', X_train.shape[0])
print('Количество образцов в тестовой выборке:', X_test.shape[0])

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
print('Среднее по каждому признаку:', X_train.mean(axis=0))
print('Отклонение по каждому признаку:', X_train.std(axis=0))

knn_clf = KNeighborsClassifier(n_neighbors=5)
knn_clf.fit(X_train, Y_train)

Y_predict_knn = knn_clf.predict(X_test)
print(classification_report(Y_test, Y_predict_knn, digits=3))

fpr_knn, tpr_knn, _ = roc_curve(Y_test, Y_predict_knn)
auc_knn = auc(fpr_knn, tpr_knn)

plt.figure()
plt.plot(fpr_knn, tpr_knn, color='green', label=f'AUC = {auc_knn:.3f}')
plt.plot([0, 1], [0, 1], color='red', linestyle='--')
plt.xlabel('FPR')
plt.ylabel('TPR')
plt.title('ROC-кривая (KNN)')
plt.legend()
plt.show()

cm_knn = confusion_matrix(Y_test, Y_predict_knn)

plt.figure()
sns.heatmap(cm_knn, annot=True, cmap='Purples')
plt.xlabel('Предсказанные классы')
plt.ylabel('Истинные классы')
plt.title('Матрица ошибок (KNN)')
plt.show()

nb_clf = GaussianNB()
nb_clf.fit(X_train, Y_train)

Y_predict_nb = nb_clf.predict(X_test)

print(classification_report(Y_test, Y_predict_nb, digits=3))

fpr_nb, tpr_nb, _ = roc_curve(Y_test, Y_predict_nb)
auc_nb = auc(fpr_nb, tpr_nb)

plt.figure()
plt.plot(fpr_nb, tpr_nb, color='green', label=f'AUC = {auc_nb:.3f}')
plt.plot([0, 1], [0, 1], color='red', linestyle='--')
plt.xlabel('FPR')
plt.ylabel('TPR')
plt.title('ROC-кривая (Наивный байесовский классификатор)')
plt.legend()
plt.show()

cm_nb = confusion_matrix(Y_test, Y_predict_nb)
```

```
plt.figure()
sns.heatmap(cm_nb, annot=True, cmap='Purples')
plt.xlabel('Предсказанные классы')
plt.ylabel('Истинные классы')
plt.title('Матрица ошибок (Наивный байесовский классификатор)')
plt.show()

f1_knn = f1_score(Y_test, Y_predict_knn)
f1_nb = f1_score(Y_test, Y_predict_nb)

if f1_knn > f1_nb:
    print("По критерию F1 лучше KNN.")
else:
    print("По критерию F1 лучше Наивный байесовский классификатор.")

if auc_knn > auc_nb:
    print("По критерию AUC лучше KNN.")
else:
    print("По критерию AUC лучше Наивный байесовский классификатор.")
```